

Pyspark

4. What are the common Actions in Apache Spark?

Some commonly used Actions in Apache Spark are as follows:

- `Reduce(func)`: This Action aggregates the elements of a dataset by using func function.
- `Count()`: This action gives the total number of elements in a Dataset.
- `Collect()`: This action will return all the elements of a dataset as an Array to the driver program.
- `First()`: This action gives the first element of a collection.
- `Take(n)`: This action gives the first n elements of dataset.
- `Foreach(func)`: This action runs each element in dataset through a for loop and executes function func on each element.

5. What is a SparkContext?

A [SparkContext](#) is the entry point to Spark. It is responsible for creating and managing Spark clusters. In a cluster SparkContext is

used to connect to the cluster manager that allocate resources to multiple tasks.

For any Spark program SparkContext object is created first.

Following is an example of creating spark context object in pyspark

```
from pyspark import SparkContext  
sc = SparkContext("local", "Hello World App")
```

6. What is an RDD?

Resilient Distribution Dataset (RDD) is an abstraction of data in Apache Spark. It is a distributed and resilient collection of records spread over many partitions. RDD hides the data partitioning and distribution behind the scenes. Main features of RDD are as follows:

- **Distributed:** Data in a RDD is distributed across multiple nodes.
- **Resilient:** RDD is a fault- tolerant dataset. In case of node failure, Spark can re- compute data.
- **Dataset:** It is a collection of data similar to collections in Scala.
- **Immutable:** Data in RDD cannot be modified after creation

Following is an example of an RDD —

```
from pyspark import SparkContext
sc = SparkContext("local", "count app")

# Create an RDD
words = sc.parallelize (
    ["scala",
     "java",
     "hadoop",
     "spark",
     "akka",
     "spark vs hadoop",
     "pyspark",
     "pyspark and spark"]
)
```

7. What are the main operations that can be done on a RDD in Apache Spark?

There are two main operations that can be performed on a RDD in Spark:

- **Transformation:** This is a function that is used to create a new RDD out of an existing RDD.
- **Action:** This is a function that returns a value to Driver program after running a computation on RDD.

Following is an example of performing an action on RDD —

```
from pyspark import SparkContext
sc = SparkContext("local", "count app")

# Create an RDD
words = sc.parallelize (
    ["scala",
     "java",
     "hadoop",
```

```
"spark",
"akka",
"spark vs hadoop",
"pyspark",
"pyspark and spark"]
)
counts = words.count()
print("Count is : {}".format(counts))
```

8. What is a DataFrame?

A DataFrame is a *Dataset* organized into named columns. It is conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations under the hood.

DataFrames can be constructed from a wide array of [sources](#) such as: structured data files, tables in Hive, external databases, or existing RDDs.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

# PySpark DataFrame
df = spark.createDataFrame([
    ('Lorem', date(2000, 8, 1)),

    ('Ipsum', date(2000, 6, 2)),

    ('Lorem2', date(2000, 5, 3))
], schema='NAME string, DATEdate')
```

9. What is a Dataset?

A Dataset is a distributed collection of data. Dataset is a new interface added in Spark 1.6 that provides the benefits of RDDs

(strong typing, ability to use powerful lambda functions) with the benefits of Spark SQL's optimized execution engine.

10. What are the most common Spark transformations?

Some of the most common Spark transformations include:

- `map()`: Applies a function to each element of an RDD and returns a new RDD.
- `filter()`: Returns a new RDD containing only the elements of the original RDD that satisfy a given condition.
- `reduce()`: Reduces an RDD to a single value by applying a function to each pair of elements in the RDD and returning the accumulated result.
- `join()`: Joins two RDDs on a common key and returns a new RDD containing the joined rows.

11. What are the most common Spark actions?

Some of the most common Spark actions include:

- `collect()`: Returns all the elements of an RDD as a list.
- `count()`: Returns the number of elements in an RDD.
- `first()`: Returns the first element of an RDD.

- `take()`: Returns the first `n` elements of an RDD.

12. What is caching in PySpark?

Caching in PySpark allows you to store RDDs in memory so that they can be reused efficiently. This can improve the performance of Spark applications that process the same datasets multiple times.

The default storage level will be `MEMORY_AND_DISK`. Following is an example of `cache()` —

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

# Create Dataframe
df = spark.range(1)

# Cache in memory
df.cache()
```

13. What is partitioning in PySpark?

Partitioning in PySpark divides an RDD into multiple sub-datasets that can be processed independently. This can improve the performance of Spark applications by allowing Spark to distribute the work across multiple nodes in the cluster.

14. What are broadcast variables in PySpark?

Broadcast variables in PySpark are variables that are broadcast to all the worker nodes in the cluster. This can be useful for sharing large read-only datasets with all the worker nodes. Following is an example of broadcast variable in pyspark—

```
>> from pyspark.context import SparkContext
>> sc = SparkContext('local', 'test')
>> b = sc.broadcast([1, 2, 3, 4, 5])
>> b.value
```

```
Out: [1, 2, 3, 4, 5]
```

15. What are accumulators in PySpark?

Accumulators in PySpark are variables that can be updated by multiple worker nodes in the cluster. This can be useful for aggregating data from multiple worker nodes into a single value.

16. How do you handle data skewness in PySpark?

Data skewness in PySpark occurs when an RDD is not evenly distributed across the partitions. This can lead to performance problems, as some partitions may be overloaded while others are idle.

One way to handle data skewness is to use the `repartition()` transformation to redistribute the data across the partitions. Another way to handle data skewness is to use the `mapPartitions()` transformation to apply a function to each partition independently.

17. How will you do memory tuning in Spark?

In case of memory tuning following points can help —

- Amount of memory used by objects Cost of accessing objects Overhead of Garbage Collection.
- Use data structures like Array of objects or primitives instead of Linked list or HashMap.
- Reduce the usage of nested data structures with a large number of small objects and pointers. E.g. Linked list has pointers within each node.
- It is a good practice to use numeric IDs instead of Strings for keys.

18. What is a Shuffle operation in Spark?

Shuffle operation is used in Spark to re-distribute data across multiple partitions. It is a costly and complex operation. In general a single task in Spark operates on elements in one partition.

To execute shuffle, one has to run an operation on all elements of all partitions. It is also called all-to-all operation.

19. What are the operations that can cause a shuffle in Spark?

Some of the common operations that can cause a shuffle internally in Spark are as follows:

- Repartition
- Coalesce

- GroupByKey
- ReduceByKey
- Cogroup
- Join

20. What is lazy evaluation in Apache Spark?

Apache Spark uses lazy evaluation as a performance optimization technique. In lazy evaluation as transformation is not applied immediately to a RDD. Spark records the transformations that have to be applied to a RDD. Once an Action is called, Spark executes all the transformations. Since Spark does not perform immediate execution based on transformation, it is called lazy evaluation.

21. What is the difference in `cache()` and `persist()` methods in Apache Spark?

Both `cache()` and `persist()` functions are used for persisting a RDD in memory across operations. The key difference between `persist()` and `cache()` is that in `persist()` you can specify the storage level. Where as in `cache()`, default strategy is used for persisting. The default storage strategy is `MEMORY_ONLY`.

22. How will you remove data from cache in Apache Spark?

In general, Apache Spark automatically removes the unused objects from cache. It uses **Least Recently Used (LRU)** algorithm to

drop old partitions. There are automatic monitoring mechanisms in Spark to monitor cache usage on each node.

In case one wants to forcibly remove an object from cache in Apache Spark `RDD.unpersist()` method can be used.

23. What is the use of SparkContext in Apache Spark?

SparkContext is the central object in Spark that coordinates different Spark applications in a cluster. In a cluster you can use SparkContext to connect to multiple Cluster Managers that allocate resources to multiple applications. For any Spark program you first create SparkContext object.

24. How will you minimize data transfer while working with Apache Spark?

Generally Shuffle operation in Spark leads to a large amount of data transfer. One shall configure Spark shuffle process for optimum data transfer using following options:

- `spark.shuffle.compress`: This configuration can be set to true to compress map output files. This reduces the amount of data transfer due to compression.
- `ByKey` operations: Minimizes the use of ByKey operations to minimize the shuffle calls.

25. What are different Persistence levels in Apache Spark?

Different Persistence levels in Apache Spark are as follows:

- **MEMORY_ONLY**: In this level, RDD object is stored as a de-serialized Java object in JVM. If an RDD doesn't fit in the memory, it will be recomputed.
- **MEMORY_AND_DISK**: In this level, RDD object is stored as a de-serialized Java object in JVM. If an RDD doesn't fit in the memory, it will be stored on the Disk.
- **MEMORY_ONLY_SER**: In this level, RDD object is stored as a serialized Java object in JVM. It is more efficient than de-serialized object.
- **MEMORY_AND_DISK_SE**: In this level, RDD object is stored as a serialized Java object in JVM. If an RDD doesn't fit in the memory, it will be stored on the Disk.
- **DISK_ONLY**: In this level, RDD object is stored only on Disk.

Pyspark Scenario based queries

1. Question: Working with CSV Files

Scenario: You have a CSV file named "data.csv" with the following columns: "id", "name", "age", and "salary". Load the data into a PySpark DataFrame and display the first 5 rows.

```
from pyspark.sql import SparkSession

# Initialize SparkSession

spark = SparkSession.builder.appName("InterviewQuestions").getOrCreate()

# Load CSV data into DataFrame

df = spark.read.csv("data.csv", header=True, inferSchema=True)

# Display the first 5 rows

df.show(5)
```

2. Question: Filtering and Aggregating Data

Scenario: You have a DataFrame "df" with columns "product_name" and "price". Filter the DataFrame to show only the products with a price greater than 1000 and calculate the average price for each product category.

```
from pyspark.sql import functions as F

# Filter products with price > 1000

filtered_df = df.filter(df["price"] > 1000)

# Calculate average price for each product category
```

```
average_price_df =
filtered_df.groupBy("product_name").agg(F.avg("price").alias("avg_price"))

average_price_df.show()
```

3. Question: Handling Missing Data

Scenario: Your DataFrame "df" contains columns "product_name", "quantity", and "price". However, some rows have missing values for the "quantity" column. Replace the missing values with 0 and calculate the total revenue for each product.

```
# Replace missing "quantity" values with 0
df = df.fillna(0, subset=["quantity"])

# Calculate total revenue for each product
df = df.withColumn("total_revenue", df["quantity"] * df["price"])

df.show()
```

4. Question: Working with Dates

Scenario: Your DataFrame "df" contains a column "transaction_date" in string format (YYYY-MM-DD). Convert this column to a DateType and calculate the total revenue generated on each date.

```
from pyspark.sql.functions import to_date

# Convert "transaction_date" to DateType
df = df.withColumn("transaction_date", to_date(df["transaction_date"], "yyyy-MM-dd"))

# Calculate total revenue for each date
daily_revenue = df.groupBy("transaction_date").agg(F.sum("quantity" *
"price").alias("total_revenue"))
```

```
daily_revenue.show()
```

5. Question: Window Functions and Ranking

Scenario: You have a DataFrame "df" with columns "product_name", "quantity", and "transaction_date". Calculate the rank of each product based on the total quantity sold, and display the top 5 products with the highest quantity.

```
from pyspark.sql import Window

# Define the Window specification

window_spec = Window.partitionBy("product_name").orderBy(F.desc("quantity"))

# Calculate rank of each product based on quantity

df = df.withColumn("rank", F.rank().over(window_spec))

# Display top 5 products with highest quantity

top_products = df.filter(F.col("rank") <= 5)

top_products.show()
```

6. Question: Joining DataFrames

Scenario: You have two DataFrames "df1" and "df2", both containing columns "customer_id" and "customer_name". Perform a full outer join between the DataFrames and display the rows where the customer names match.

```
# Perform full outer join between df1 and df2

joined_df = df1.join(df2, on="customer_id", how="full_outer")

# Filter rows where customer names match

matched_customers = joined_df.filter(df1["customer_name"] == df2["customer_name"])
```

```
matched_customers.show()
```

7. Question: Broadcast Join

Scenario: You have a large DataFrame "big_df" with columns "customer_id" and "order_total", and a small DataFrame "small_df" with columns "customer_id" and "customer_name". Use broadcast join to join the DataFrames and display the customer names along with their order totals.

```
# Broadcast join small_df with big_df

from pyspark.sql.functions import broadcast

joined_df = big_df.join(broadcast(small_df), on="customer_id", how="inner")

# Display customer names with order totals

joined_df.select("customer_name", "order_total").show()
```

8. Question: Working with UDFs

Scenario: You have a DataFrame "df" with a column "text" containing sentences. Create a User-Defined Function (UDF) to calculate the average length of sentences, and apply the UDF to create a new column "avg_length".

```
# User-Defined Function

from pyspark.sql.functions import udf

from pyspark.sql.types import IntegerType

def calculate_avg_length(text):

    sentences = text.split(".")

    total_length = sum(len(sentence) for sentence in sentences)

    avg_length = total_length / len(sentences)

    return int(avg_length)
```

```
# Register UDF

avg_length_udf = udf(calculate_avg_length, IntegerType())

# Apply UDF to create "avg_length" column

df = df.withColumn("avg_length", avg_length_udf("text"))

df.show()
```

9. Question: Handling Large Data

Scenario: You have a very large DataFrame "big_df" with millions of rows. Implement a sampling technique to select a random sample of 1% of the data and display the sampled DataFrame.

```
# Sample 1% of the data

sampled_df = big_df.sample(withReplacement=False, fraction=0.01)

sampled_df.show()
```

10. Question: Data Validation

Scenario: You have a DataFrame "df" with columns "age" and "gender". Validate the data to ensure that the "age" column does not contain negative values, and the "gender" column only contains 'Male' or 'Female'. Display the rows that fail the validation.

```
# Data Validation

invalid_data = df.filter((df["age"] < 0) | (~df["gender"].isin(['Male',
'Female'])))

invalid_data.show()
```

11. Question: Cross-Joining DataFrames

Scenario: You have two DataFrames "df1" and "df2", both containing columns "product_name" and "quantity". Perform a cross-join between the DataFrames to get all possible combinations of products and quantities.

```
# Perform cross-join between df1 and df2
```

```
cross_joined_df = df1.crossJoin(df2)
```

```
cross_joined_df.show()
```

12. Question: Handling Duplicate Data

Scenario: Your DataFrame "df" contains duplicate rows. Remove the duplicate rows based on all columns and display the DataFrame without duplicates.

```
# Remove duplicate rows based on all columns
```

```
deduplicated_df = df.dropDuplicates()
```

```
deduplicated_df.show()
```

13. Question: Exploratory Data Analysis

Scenario: You have a DataFrame "df" with columns "age" and "income". Perform exploratory data analysis to understand the distribution of ages and income using summary statistics.

```
# Exploratory Data Analysis
```

```
summary_stats = df.select(F.mean("age"), F.min("age"), F.max("age"),  
F.mean("income"), F.min("income"), F.max("income"))
```

```
summary_stats.show()
```

14. Question: Handling Outliers

Scenario: Your DataFrame "df" contains a column "income". Use the Interquartile Range (IQR) method to detect and handle outliers in the "income" column.

```
# Handling Outliers using IQR method
```

```

q1, q3 = df.approxQuantile("income", [0.25, 0.75], 0.01)

iqr = q3 - q1

lower_bound = q1 - 1.5 * iqr

upper_bound = q3 + 1.5 * iqr

df = df.filter((df["income"] >= lower_bound) & (df["income"] <= upper_bound))

df.show()

```

15. Question: Applying Machine Learning

Scenario: You have a DataFrame "df" with features "age", "income", and a label "target" indicating whether a customer made a purchase (1) or not (0). Split the data into training and testing sets, and train a logistic regression model to predict the "target" based on the features.

```

from pyspark.ml.classification import LogisticRegression

from pyspark.ml.feature import VectorAssembler

from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Prepare features and label columns

feature_cols = ["age", "income"]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

df = assembler.transform(df)

# Split data into training and testing sets

train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)

# Train a logistic regression model

lr = LogisticRegression(labelCol="target", featuresCol="features")

```

```

model = lr.fit(train_data)

# Make predictions on test data

predictions = model.transform(test_data)

# Evaluate model performance

evaluator = BinaryClassificationEvaluator(labelCol="target")

auc = evaluator.evaluate(predictions)

print("AUC:", auc)

```

16. Question: Working with Nested Data

Scenario: Your DataFrame "df" contains a column "address" that contains nested data in the form of a struct with fields "city" and "zipcode". Extract the "city" and "zipcode" from the "address" column and create separate columns for them.

```

# Extract "city" and "zipcode" from "address" struct

df = df.withColumn("city", df["address.city"])

df = df.withColumn("zipcode", df["address.zipcode"])

df.show()

```

17. Question: Handling Imbalanced Data

Scenario: Your DataFrame "df" contains a column "target" with binary values (0 or 1), indicating whether a customer made a purchase (1) or not (0). The data is imbalanced, with a small number of positive samples. Implement the Synthetic Minority Over-sampling Technique (SMOTE) to balance the data and train a classification model.

```

from imblearn.over_sampling import SMOTE

from pyspark.ml.feature import VectorAssembler

```

```
# Prepare features and label columns

feature_cols = ["age", "income"]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

df = assembler.transform(df)


# Implement SMOTE to balance the data

smote = SMOTE(sampling_strategy='auto', random_state=42)

features_resampled, target_resampled = smote.fit_resample(df.select("features"),
df.select("target"))


# Create a DataFrame with resampled data

balanced_df = spark.createDataFrame(zip(features_resampled, target_resampled),
["features", "target"])


# Split data into training and testing sets

train_data, test_data = balanced_df.randomSplit([0.8, 0.2], seed=42)


# Train a Logistic regression model

lr = LogisticRegression(labelCol="target", featuresCol="features")

model = lr.fit(train_data)


# Make predictions on test data

predictions = model.transform(test_data)


# Evaluate model performance

evaluator = BinaryClassificationEvaluator(labelCol="target")

auc = evaluator.evaluate(predictions)
```

```
print("AUC:", auc)
```

18. Question: Handling Large Text Data

Scenario: You have a DataFrame "df" with a column "text" containing large text data. Implement feature extraction techniques such as TF-IDF and train a text classification model.

```
from pyspark.ml.feature import Tokenizer, StopWordsRemover, CountVectorizer, IDF
```

```
from pyspark.ml.classification import NaiveBayes
```

```
from pyspark.ml import Pipeline
```

```
# Tokenize text data
```

```
tokenizer = Tokenizer(inputCol="text", outputCol="words")
```

```
df = tokenizer.transform(df)
```

```
# Remove stop words
```

```
remover = StopWordsRemover(inputCol="words", outputCol="filtered_words")
```

```
df = remover.transform(df)
```

```
# Calculate Term Frequency (TF)
```

```
cv = CountVectorizer(inputCol="filtered_words", outputCol="raw_features")
```

```
model = cv.fit(df)
```

```
df = model.transform(df)
```

```
# Calculate Inverse Document Frequency (IDF)
```

```
idf = IDF(inputCol="raw_features", outputCol="features")
```

```
idf_model = idf.fit(df)
```

```
df = idf_model.transform(df)
```

```

# Split data into training and testing sets

train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)


# Train a Naive Bayes classification model

nb = NaiveBayes(labelCol="target", featuresCol="features")

model = nb.fit(train_data)


# Make predictions on test data

predictions = model.transform(test_data)


# Evaluate model performance

evaluator = BinaryClassificationEvaluator(labelCol="target")

auc = evaluator.evaluate(predictions)

print("AUC:", auc)

```

19. Question: Data Serialization

Scenario: You have a large DataFrame "df" that needs to be shared with other teams in a serialized format. Serialize the DataFrame to a Parquet file for efficient storage and distribution.

```

# Serialize DataFrame to Parquet file

df.write.parquet("data.parquet")

```

20. Question: Working with Avro Data

Scenario: You have an Avro file named "data.avro" containing customer data in binary format. Load the Avro data into a PySpark DataFrame for further processing and analysis.

```

# Load Avro data into DataFrame

```

```
df = spark.read.format("avro").load("data.avro")

df.show()
```

21. Question: Broadcast Variables

Scenario: You have a large DataFrame "df" and a small list of values that you want to use for filtering the DataFrame. Use broadcast variables to efficiently join the DataFrame with the list and display the results.

```
# Small list of values
filter_values = [10, 20, 30]

# Broadcast the small list
broadcast_values = spark.sparkContext.broadcast(filter_values)

# Filter the DataFrame using broadcast variable
filtered_df = df.filter(df["column_name"].isin(broadcast_values.value))

filtered_df.show()
```

23. Question: Working with Nested JSON Data

Scenario: You have a JSON file named "data.json" containing nested data. Load the JSON data into a PySpark DataFrame and extract specific fields from the nested structure.

```
# Load JSON data into DataFrame
df = spark.read.json("data.json")

# Extract specific fields from nested structure
df = df.select("outer_field.inner_field1", "outer_field.inner_field2")
```

```
df.show()
```

24. Question: Cross-Validation for Model Selection

Scenario: You have a DataFrame "df" with features and a binary label. Perform k-fold cross-validation to evaluate the performance of multiple classification models and select the best-performing one.

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

from pyspark.ml.evaluation import BinaryClassificationEvaluator

from pyspark.ml.classification import LogisticRegression, RandomForestClassifier,
GBTClassifier

# Prepare features and label columns

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

df = assembler.transform(df)

# Initialize classifiers

lr = LogisticRegression(labelCol="label", featuresCol="features")

rf = RandomForestClassifier(labelCol="label", featuresCol="features")

gbt = GBTClassifier(labelCol="label", featuresCol="features")

# Create a parameter grid for each classifier

param_grid_lr = ParamGridBuilder().build()

param_grid_rf = ParamGridBuilder().build()

param_grid_gbt = ParamGridBuilder().build()

# Initialize evaluator

evaluator = BinaryClassificationEvaluator()
```

```
# Perform k-fold cross-validation for each classifier
```

```
cv_lr = CrossValidator(estimator=lr, estimatorParamMaps=param_grid_lr,  
evaluator=evaluator, numFolds=5)
```

```
cv_rf = CrossValidator(estimator=rf, estimatorParamMaps=param_grid_rf,  
evaluator=evaluator, numFolds=5)
```

```
cv_gbt = CrossValidator(estimator=gbt, estimatorParamMaps=param_grid_gbt,  
evaluator=evaluator, numFolds=5)
```

```
# Fit the models
```

```
cv_lr_model = cv_lr.fit(df)
```

```
cv_rf_model = cv_rf.fit(df)
```

```
cv_gbt_model = cv_gbt.fit(df)
```

```
# Get the best model for each classifier
```

```
best_lr_model = cv_lr_model.bestModel
```

```
best_rf_model = cv_rf_model.bestModel
```

```
best_gbt_model = cv_gbt_model.bestModel
```

```
print("Best Logistic Regression Model:", best_lr_model)
```

```
print("Best Random Forest Model:", best_rf_model)
```

```
print("Best Gradient Boosting Tree Model:", best_gbt_model)
```